

哈尔滨工业大学

编译系统实验报告（三）

姓名	李凡丁
学号	7203610625
学院	计算机科学与技术
教师	朱庆福

2023 年 5 月

一、实验目的

1. 巩固对中间代码生成的基本功能和原理的认识。
2. 能够基于语法制导翻译的知识进行中间代码生成。
3. 掌握类高级语言中基本语句所对应的语义动作。

二、实验环境

1. Ubuntu 20.04
2. GCC 9.4.0
3. Flex 2.6.4
4. Bison 3.5.1

三、实验过程

1. 中间代码的作用：

经过了词法分析、语法分析和语义分析，我们得到了一棵语法分析树，接下来我们将以此为输入进行源代码的翻译，中间代码是源代码和目标代码之间的桥梁，它会将复杂的高级语言翻译成简单并且易懂的三地址码。然后后续将以这个三地址码为进一步分析的依据，生成目标代码。

2. 翻译模式：

我们将程序中不同的语句进行分类，包括基本表达式，基本语句，声明语句，条件语句，调用语句，数据和结构体的赋值语句等语句。我们为每种语句，每个主要的语法单元设计相应的翻译函数。

3. 实验中用到的数据结构：

本实验中我们需要在实验 2 的基础上额外实现两个数据结构：分别是操作数结构体以及中间代码结构体。前者的作用是保存每个操作数的类型和数值，用来完成中间代码的转换；后者的作用是保存生成的中间代码方便输出。

```
struct Operand_  
{  
    enum OperandKind kind;  
    int u_int;  
    char *u_char;  
    Type type;  
};
```

图表 1：操作数结构体

```

struct InterCode_
{
    enum InterCodeKind kind;
    union
    {
        // LABEL, FUNCTION, GOTO, RETURN, ARG, PARAM, READ, WRITE
        struct
        {
            Operand op;
        } operand_1;
        // ASSIGN, CALL, ADDRESS1, ADDRESS2, ADDRESS3
        struct
        {
            Operand op1, op2;
        } operand_2;
        // ADD, SUB, MUL, DIV
        struct
        {
            Operand result, op1, op2;
        } operand_3;
    };
};

// IF
struct
{
    Operand x, relop, y, z;
} operand_if;
// DEC
struct
{
    Operand op;
    int size;
} operand_dec;
};
InterCode prev;
InterCode next;
};

```

图表 2：中间代码结构体

其中中间代码这个结构体中，我采用了双向链表来进行前后节点之间的关联。

4. 翻译过程：

全部的语法单元以及基本表达式等内容均对应了一个翻译函数。我们在定义操作的时候先定义一个操作数，然后赋予字段值。然后我们调用中间代码生成函数来生成中间代码，并将其进行存储，利用上文中提到的结构体。然后在最后进行统一输出即可。

下图分别为中间代码生成函数以及一个翻译函数的例子。

```

InterCode create_intercode(enum InterCodeKind kind, int intercode_type, ...
{
    InterCode ic = (InterCode)malloc(sizeof(struct InterCode_));
    ic->kind = kind;
    va_list valist;
    va_start(valist, (intercode_type % 10));

    if (intercode_type == INTERCODE_1)
    {
        ic->operand_1.op = va_arg(valist, Operand);
    }
    else if (intercode_type == INTERCODE_2)
    {
        ic->operand_2.op1 = va_arg(valist, Operand);
        ic->operand_2.op2 = va_arg(valist, Operand);
    }
    else if (intercode_type == INTERCODE_3)
    {
        ic->operand_3.result = va_arg(valist, Operand);
        ic->operand_3.op1 = va_arg(valist, Operand);
        ic->operand_3.op2 = va_arg(valist, Operand);
    }
    else if (intercode_type == INTERCODE_IF)
    {
        ic->operand_if.x = va_arg(valist, Operand);
        ic->operand_if.relop = va_arg(valist, Operand);
        ic->operand_if.y = va_arg(valist, Operand);
        ic->operand_if.z = va_arg(valist, Operand);
    }
    else if (intercode_type == INTERCODE_DEC)
    {
        ic->operand_dec.op = va_arg(valist, Operand);
        ic->operand_dec.size = va_arg(valist, int);
    }

    va_end(valist);
    return ic;
}

```

图表 3：中间代码生成函数

```

void translate_CompSt(Treenode now)
{
    // CompSt → LC DefList StmtList RC
    Treenode deflist = now->firstchild->nxtbro;
    Treenode stmtlist = deflist->nxtbro;
    translate_DefList(deflist);
    translate_StmtList(stmtlist);
    return;
}

```

图表 4：比较语句的翻译过程

在这段代码中，我们首先对于语句进行成分划分（见函数第一行注释），然后我们

在语法分析树中将这部分依次找到，然后将能直接进行翻译的部分进行翻译，不能直接翻译的部分进行递归翻译即可。

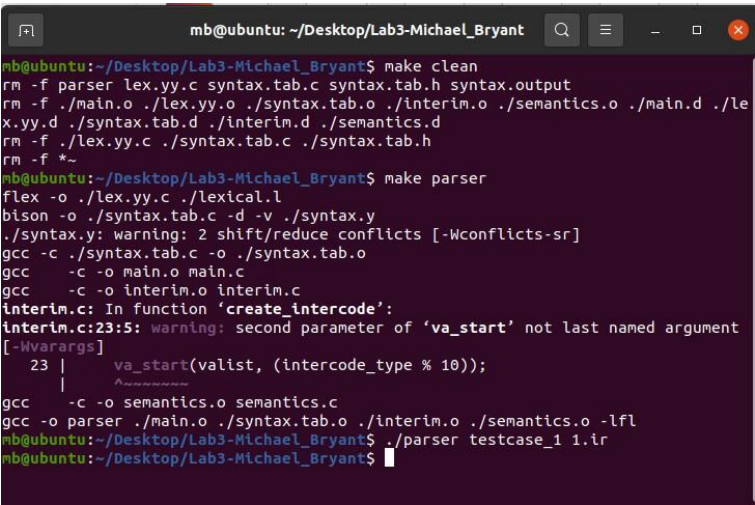
四、 实验的操作流程

在本实验中，我仍然编写了 MakeFile 文件进行实验的运行。

首先采用 make parser 命令生成中间代码生成器。

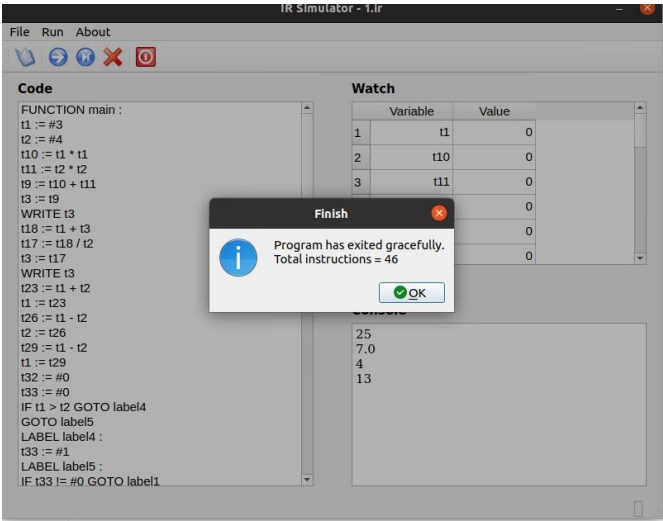
然后使用命令 ./parser test1.cmm test1.ir 生成对于 test1.cmm 代码的中间代码翻译，并存储在 test1.ir 文件中，最终使用 python irsim.py 命令运行小程序虚拟机，并利用其检验中间代码生成的正确性。命令运行的结果，生成的中间代码在虚拟机小程序中的运行情况均在下一部分中进行体现。

五、 实验的运行结果



```
mb@ubuntu:~/Desktop/Lab3-Michael_Bryant$ make clean
rm -f parser lex.yy.c syntax.tab.c syntax.tab.h syntax.output
rm -f ./main.o ./lex.yy.o ./syntax.tab.o ./interim.o ./semantics.o ./main.d ./lex.yy.d ./syntax.tab.d ./interim.d ./semantics.d
rm -f ./lex.yy.c ./syntax.tab.c ./syntax.tab.h
rm -f *~
mb@ubuntu:~/Desktop/Lab3-Michael_Bryant$ make parser
flex -o ./lex.yy.c ./lexical.l
bison -o ./syntax.tab.c -d -v ./syntax.y
./syntax.y: warning: 2 shift/reduce conflicts [-Wconflicts-sr]
gcc -c ./syntax.tab.c -o ./syntax.tab.o
gcc -c -o main.o main.c
gcc -c -o interim.o interim.c
interim.c: In function 'create_intercode':
interim.c:23:5: warning: second parameter of 'va_start' not last named argument [-Wvarargs]
   23 |     va_start(valist, (intercode_type % 10));
      |     ~~~~~^
gcc -c -o semantics.o semantics.c
gcc -o parser ./main.o ./syntax.tab.o ./interim.o ./semantics.o -lfl
mb@ubuntu:~/Desktop/Lab3-Michael_Bryant$ ./parser testcase_1 1.ir
mb@ubuntu:~/Desktop/Lab3-Michael_Bryant$
```

图表 5：命令执行结果



IR Simulator - 1.ir

File Run About

Code

```
FUNCTION main:
t1 := #3
t2 := #4
t10 := t1 * t1
t11 := t2 * t2
t9 := t10 + t11
t3 := t9
WRITE t3
t18 := t1 + t3
t17 := t18 / t2
t3 := t17
WRITE t3
t23 := t1 + t2
t1 := t23
t26 := t1 - t2
t2 := t26
t29 := t1 - t2
t1 := t29
t32 := #0
t33 := #0
IF t1 > t2 GOTO label4
GOTO label5
LABEL label4:
t33 := #1
LABEL label5:
IF t33 != #0 GOTO label1
```

Watch

Variable	Value
t1	0
t10	0
t11	0
t9	0
t3	0
t18	0
t17	0
t23	0
t1	0
t26	0
t2	0
t29	0
t1	0
t32	0
t33	0
t25	7.0
t4	4
t13	13

Finish

Program has exited gracefully.
Total instructions = 46

OK

图表 6：样例 1 运行结果

六、 实验总结

通过本次实验，我对于中间代码的生成过程，实现方法有了进一步的认识，并且在小程序执行中间代码的运行过程中深化了中间代码在编译系统中桥梁的作用。